

Laboratorio 1

Esquemas de comunicación e introducción a Wireshark

Integrantes:

231010 – Dilary Sarahí Cruz López

231135 - Javier Alexander Linares Chang

Primera parte: Esquemas de comunicación

3.1 - Transmisión de códigos

Emisor	Esquema	Mensaje Enviado	Mensaje Recibido	Errores
Javier	Morse	Como te llamas	kjtjnellamt	7
Dilary	Morse	Que horas son	quehorasson	0
Javier	Baudot	Redes es difícil	redesesuifcil	1
Dilary	Baudot	Holaa Mundo	holaimundo	1
Javier	Morse	Baudot y Morse	baudotcosrse	3
Dilary	Baudot	Practicando	practicaddo	1

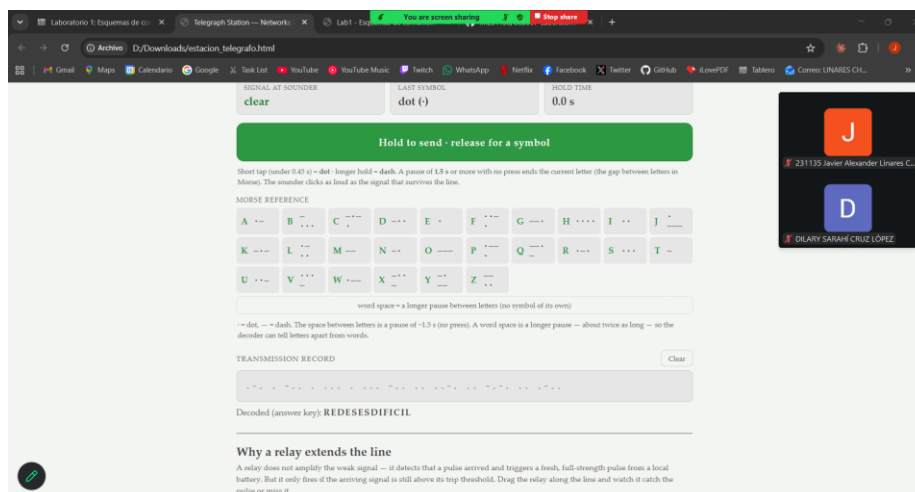


Imagen 2: Estación de telégrafo virtual en modo Morse, mostrando la señal recibida y la decodificación del mensaje "Redes es difícil".

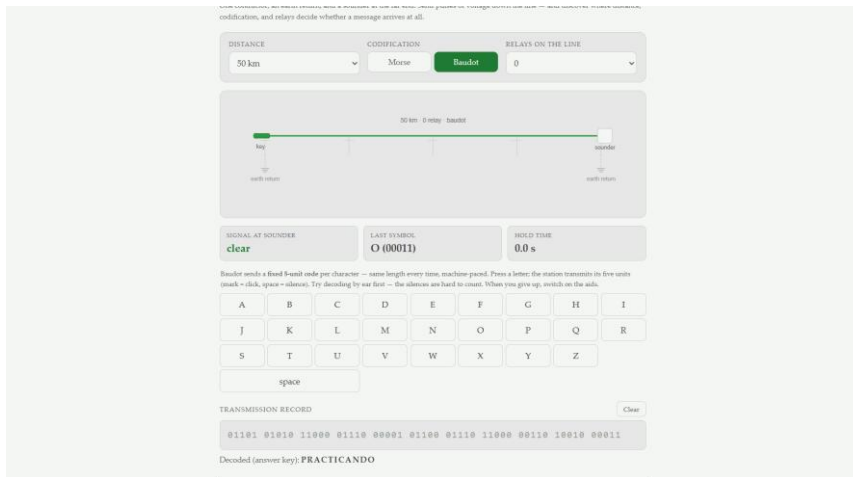


Imagen 3: Estación de telégrafo virtual en modo Baudot, enviando el mensaje "Practicando" como una secuencia de bits de 5 unidades por letra.

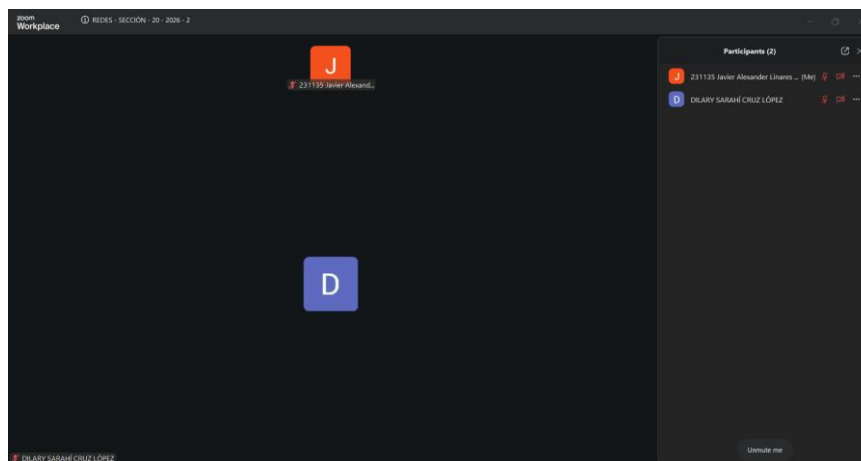


Imagen 4: Llamada de Zoom utilizada para los mensajes en Morse y Baudot durante la parte 3.1.

¿Qué esquema fue más fácil y cuál más difícil de transmitir?

Para nosotros el código Morse se sintió más fácil de transmitir, porque solo hay que manejar pulsos cortos y largos (punto y raya) y no hay que estar armando una secuencia exacta de bits. Con Baudot en cambio hay que completar cada letra con sus 5 unidades fijas, y si uno se atrasa o se adelanta un poco al contar los espacios ya se rompe toda la letra, entonces para nosotros Baudot fue el esquema más difícil de transmitir, aunque una vez agarrada la práctica se volvía más mecánico.

¿Con cuál esquema hubo menos errores?

Aunque Morse nos pareció más fácil de enviar, fue el que terminó con más errores: de los tres mensajes que mandamos en Morse uno salió perfecto ("Que horas son"), pero los otros dos llegaron bastante alterados, "Como te llamas" se recibió como "kjtjnellamt" (7 caracteres distintos comparando letra por letra) y "Baudot y Morse" se recibió como "baudotcorse" (3 caracteres distintos). En cambio los tres mensajes en Baudot llegaron casi intactos, con solo 1 error de una letra en cada uno ("redesesuificil", "holaimundo", "practicaddo"). En total Morse tuvo 10 errores entre

los tres mensajes y Baudot solo 3, por lo que para nosotros Baudot terminó siendo el esquema con menos errores, aunque haya sido el más difícil de transmitir.

3.2 - Transmisión “empaquetada”

Esquema seleccionado para todos los mensajes: Morse

Emisor	Esquema	Mensaje Enviado	Mensaje Recibido	Errores
Javier	Morse	Notas de voz	notasdevog	1
Dilary	Morse	Enviando codigo	enviandocobigo	1
Javier	Morse	Esto es dificil	ehtoesdificil	1
Dilary	Morse	Whatsapp si	whatsapphi	1
Javier	Morse	Grabando xd	grabandoxd	0
Dilary	Morse	Escuchando no	escuchandodo	1

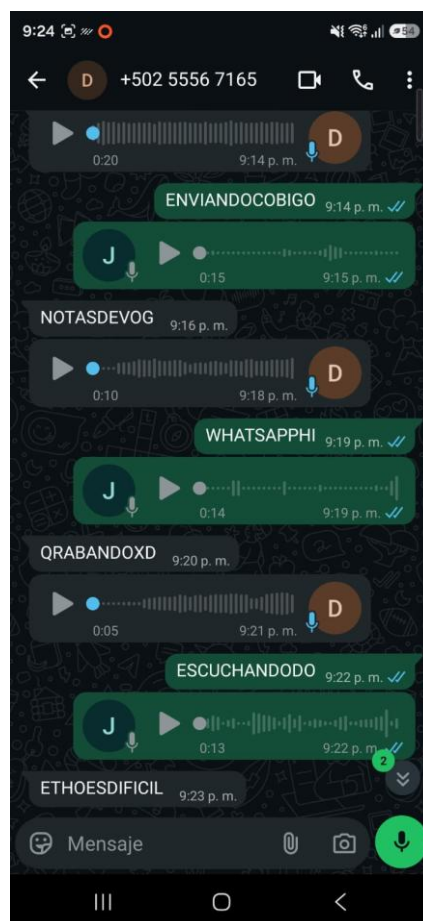


Imagen 5: Conversación de WhatsApp donde se enviaron y recibieron las notas de voz con los mensajes en código Morse de la parte 3.2.

¿Qué dificultades involucra el enviar un mensaje de esta forma "empaquetada"?

Con la transmisión empaquetada la principal dificultad es que se pierde la retroalimentación inmediata que sí había estando en la misma llamada: si el receptor no entendía algo tenía que esperar a que se diera cuenta y pedir que se repitiera el

mensaje completo, en vez de simplemente decir "repite" en el momento. También cuesta más decodificar, porque hay que reproducir la nota de voz varias veces y contar los pulsos solo de oído, sin ver la señal en pantalla como en la estación de telégrafo, y cualquier ruido de fondo o corte en el audio puede hacer que se pierda un pulso. A favor sí notamos que, como uno puede grabar de nuevo si se equivoca antes de mandar el mensaje, los errores terminaron siendo más pequeños que en la transmisión en vivo: de los 6 mensajes que mandamos, 5 llegaron con un solo carácter mal y 1 llegó perfecto ("Grabando xd"), muy por debajo de los errores que tuvimos en Morse durante la 3.1.

3.3 - Conmutación de mensajes

Para esta parte trabajamos junto con la otra pareja del curso, conformada por Cristian Tunches (231359) y Daniel Chet (23117), entre los cuatro nos organizamos según la topología de la Imagen 3: tres clientes y un conmutador en medio.

Protocolo utilizado en la parte 3.3 (conmutación)

Para la parte de conmutación lo que acordamos entre los cuatro fue un protocolo bastante simple, básicamente basado en decir en voz alta para quién era el mensaje, preguntar si el conmutador podía recibir, y que el conmutador se encargara de guardar y reenviar cada nota de voz (esto es un esquema de store-and-forward). Los pasos que seguimos fueron los siguientes:

1. **Direccionamiento.** Antes de mandar la nota de voz con el mensaje en código, el que enviaba decía en claro para quién era el mensaje, por ejemplo "De A para C", y eso le servía al conmutador como una especie de cabecera para saber el origen y el destino sin tener que escuchar el mensaje en código todavía.
2. **Control de disponibilidad.** Para no saturar al conmutador, el cliente primero preguntaba "¿Listo?" antes de empezar a grabar. Si el conmutador estaba libre respondía "dale" y ahí sí se mandaba el mensaje, pero si estaba ocupado reenviando otra nota respondía "espera" y el cliente tenía que esperar unos segundos y volver a preguntar. Con esto nos aseguramos de que el conmutador nunca estuviera atendiendo dos cosas a la vez.
3. **Envío y reenvío.** Ya con el "dale", el emisor mandaba la nota de voz con el mensaje codificado, el conmutador la recibía y después la reenviaba hacia el destino, manteniendo siempre el "De A para C" al inicio para que el que la recibía supiera de quién venía.

4. **Orden de atención.** Cuando dos clientes pedían enviar casi al mismo tiempo, el conmutador simplemente le daba prioridad al que había hablado primero y al otro le decía "espera", entonces las peticiones se iban resolviendo en el orden en que iban llegando.

¿Qué posibilidades incluye la introducción de un conmutador en el sistema?

Meter un conmutador en el sistema abre la posibilidad de que los clientes no tengan que conectarse directamente entre todos ellos. Si no existiera el conmutador cada cliente necesitaría un canal directo con cada uno de los demás, y eso crece muchísimo entre más gente se agregue; con el conmutador en medio basta con que cada cliente tenga una sola conexión hacia él y el conmutador se encarga de reenviar hacia el destino correcto. Esto también permite sumar más clientes sin tener que rehacer toda la red, y que el conmutador funcione como un punto donde se puede controlar el orden en que se atienden los mensajes, como hicimos nosotros con el "listo/espera".

¿Qué ventajas/desventajas se tienen al momento de agregar más conmutadores al sistema?

Agregar más conmutadores tiene la ventaja de que si uno falla o está saturado los mensajes pueden seguir su camino por otro, entonces la red deja de depender de un solo punto y en teoría se pueden atender más clientes al mismo tiempo porque cada conmutador se encarga de una parte del tráfico. La desventaja es que ahora hay que coordinar entre los conmutadores, por ejemplo cómo se pasan los mensajes de un conmutador a otro cuando el cliente destino no está conectado al mismo conmutador que el emisor, y eso implica acordar un protocolo más complejo que el simple "listo/espera" que usamos nosotros, porque ya no basta con saber si el conmutador propio está libre sino también si el siguiente conmutador en la ruta lo está. En pocas palabras, se gana en redundancia y capacidad, pero se pierde algo de simplicidad.

Conclusiones

Con esta práctica nos dimos cuenta de que la facilidad para transmitir un mensaje y la cantidad de errores que produce no siempre van de la mano: en nuestro caso Morse se sintió más natural de enviar, pero fue el esquema con más errores, mientras que Baudot costó más armarlo bien, pero llegó casi perfecto las tres veces. Esto nos deja claro que un esquema de codificación más rígido, con una estructura fija como los 5 bits de Baudot, tiende a ser más confiable, aunque sea menos intuitivo de usar a mano.

También notamos que pasar de una comunicación en vivo a una empaquetada (por nota de voz) le quita al sistema la posibilidad de corregir sobre la marcha, lo que obliga a ser más cuidadoso al momento de codificar el mensaje porque ya no hay

manera de aclarar dudas en el momento; aun así, la posibilidad de grabar antes de enviar hizo que los errores fueran más pequeños que en la transmisión en vivo.

Por último, con la parte de conmutación entendimos por qué en las redes reales casi nunca se conecta cada dispositivo con todos los demás, sino que se usa un intermediario que organiza el tráfico, y que agregar más de esos intermediarios trae ventajas de redundancia pero también obliga a definir protocolos más elaborados para que los mensajes lleguen bien de un conmutador a otro.